REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Ignacio Cofré Sandoval

2 de abril de 2026

19:26

1. Entorno experimental

1.1. Especificaciones del sistema

- **Sistema operativo:** EndeavourOS x64.
- **Procesador:** Intel i3-N300 3.8GHz.
- **Ram:** 16GB.
- **Compilador:** g++ 15.2.1.
- **Flags:** -std=c++17, -Wall, -Wextra, -g y -O2

1.2. Datasets

Los datasets utilizados en este experimento siguen el siguiente formato tanto para el analisis de algoritmos de ordenamiento y algoritmos de multiplicación de matrices.

Indice	Tamaño	Tiempo [ns]	Memoria	algoritmo
0	...	...	...	...
1	...	...	...	...
...	...	...	...	...

Fue necesario realizar una limpieza del dataset previo a la generacion de los graficos, ya que el metodo empleado para la ejecucion de las pruebas y toma de datos de las mismas genero en el archivo de salida caracteres que al momento de ser tratados generaban filas vacias las cuales debian ser removidas.

2. Resultados y Análisis

2.1. Algoritmos de ordenamiento

Introduccion: Con el fin de poder verificar de forma experimental las complejidades asociadas a algoritmos de ordenamiento tales como ***merge-sort***, ***quick-sort*** y ***std::sort***(libreria estandar de c++), llevamos a cabo mediciones tanto de tiempo de ejecucion como uso de memoria de los algoritmos en cuestion para distintos tamaños de entrada (10^1, 10^3, 10^5 y 10^7), para luego poder generar graficos y analizar los resultados con el objetivo de concluir si el comportamiento de los algoritmos es el esperado segun sus complejidades teoricas.

2.1.1. Analisis de tiempo

	Tamaño entrada	Algoritmo	Tiempo medio de ejecucion	Desviacion estandar	Cantidad de muestras
0	10	merge	6628.888889	714.634492	18
1	10	quick	4066.222222	989.871623	18
2	10	std::sort	4482.611111	1134.080044	18
3	1000	merge	378904.944444	33217.083191	18
4	1000	quick	79061.888889	19006.587892	18
5	1000	std::sort	48214.555556	17686.456537	18
6	100000	merge	48551985.666667	5177993.270268	18
7	100000	quick	11722067.388889	3605245.194886	18
8	100000	std::sort	6394680.944444	2570092.521951	18
9	10000000	merge	6320678299.000000	675741581.977279	18
10	10000000	quick	1594876913.333333	420415684.535780	18
11	10000000	std::sort	881570211.888889	347984322.455967	18

Tabla 1: *Tabla comparativa de los datos obtenidos en el experimento.*

Esta tabla muestra el tiempo promedio que le tomo a cada algoritmo terminar su ejecucion para los distintos tamaños de entrada.

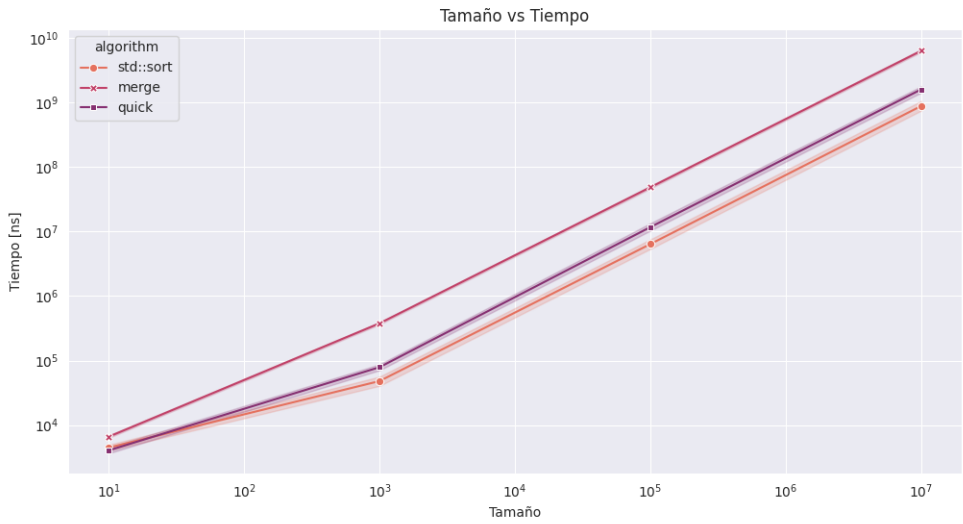

Figura 1: Relacion entre el tamaño de la entrada y tiempo de ejecucion del algoritmo para dicha entrada (escala logaritmica).

El grafico anterior permite hacer una comparacion visual entre algoritmos. Ademas este nos permite de cierta manera corroborar las complejidades teoricas determinadas para estos algoritmos de ordenamiento.
En el grafico podemos observar que el comportamiento de estos algoritmos es aproximadamente lineal en escala logaritmica lo cual es consistente con la complejidad teorica de estos, la cual para todos es $\boldsymbol{O(n \log n)}$ para el caso promedio, por lo que ver que los tres algoritmos forman lo mas cercano a una recta en una escala logaritmica es lo esperado.
A pesar de que los tres algoritmos siguen una tendencia casi lineal, podemos notar que las rectas de los algoritmos ***std::sort*** y ***quick-sort*** son bastante cercanas denotando tiempos de ejecucion promedio similares para entradas del mismo tamaño, esto debido a que la implementacion de ***std::sort*** hace uso de ***introsort*** el cual internamente hace uso de ***quicksort*** junto a otros algoritmos para evitar el peor caso de este ultimo ($\boldsymbol{O(n^2)}$) lo que explicaria la similaridad entre los tiempos promedio de ejecucion para tamaño de entradas menores y una leve superioridad de ***std::sort*** conforme el tamaño de la entrada aumenta.

Por otro lado en el caso del algoritmo ***merge-sort*** se midieron tiempos promedio de ejecucion mayores, una de la razones de este comportamiento es la naturaleza de estos algoritmos.
Mientras que ***std::sort*** y ***quick-sort*** son algoritmos de naturaleza *in-place* lo que quiere decir que realizan el ordenamiento sin necesidad de utilizar estructuras auxiliares si no que mas bien logran el ordenamiento mediante el uso de recursion, por otro lado ***merge-sort*** es de naturaleza *no in-place* lo que significa que este algoritmo hace uso de estructuras auxiliares, copia de datos a estas mismas, mas lectura y escritura de memoria. Los que genera un mayor impacto en el rendimiento.
Esto causa que si bien en la teoria para el caso promedio estos 3 algoritmos tienen la misma complejidad, al momento de realizar pruebas reales comienzan a relucir estas diferencias sutiles entre los algoritmos.

2.1.2. Analisis de memoria

	Tamaño	Algoritmo	Uso medio de memoria	Desviacion estandar	Cantidad de muestras
0	10	merge	3701.111111	60.364470	18
1	10	quick	3681.777778	75.287336	18
2	10	std::sort	3734.444444	83.670377	18
3	1000	merge	3663.555556	66.508047	18
4	1000	quick	3698.666667	64.977824	18
5	1000	std::sort	3697.555556	78.947654	18
6	100000	merge	4537.777778	67.663141	18
7	100000	quick	4277.777778	45.030128	18
8	100000	std::sort	4712.888889	71.390302	18
9	10000000	merge	82899.777778	654.233593	18
10	10000000	quick	70308.222222	548.037735	18
11	10000000	std::sort	83140.444444	573.961899	18

Tabla 2: *Tabla comparativa de los datos obtenidos en el experimento.*

Esta tabla muestra el pico promedio de uso de memoria del algoritmo tomando tambien en cuenta el parseo de las entradas.

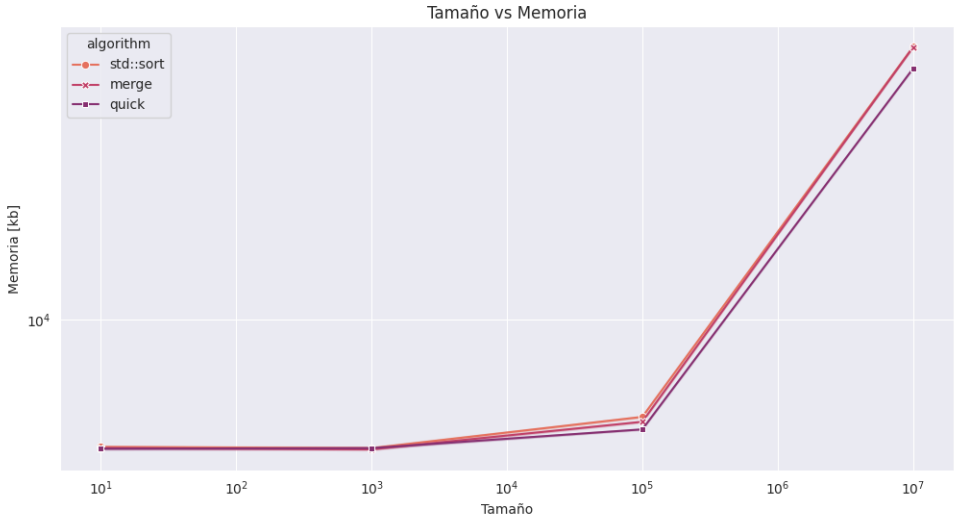

Figura 2: Grafica que relaciona tamaño de entrada con el pico de uso de memoria del algoritmo.

En el grafico anterior podemos ver como aumenta el pico del uso de memoria conforme aumenta el tamaño de la entrada.
Es importante mencionar que el metodo utilizado para poder medir el uso de memoria no nos permite realizar la medicion solo del algoritmo si no que mas bien de toda la ejecucion del programa, por lo que en estos picos de memoria tambien se esta tomando en cuenta la creacion de los arreglos de entrada a partir de archivos de texto. A pesar de esta causa de ruido en las mediciones, es posible notar como el algoritmo ***merge-sort*** luego de que las entradas son mas grandes que 10^3 el uso de memoria es levemente mayor que el de sus contra partes, esto debido a lo mencionado anteriormente sobre la naturaleza *no in-place* del algoritmo, ya que mientras mas grande es la entrada mas divisiones se deben hacer sobre el arreglo, lo que implica mas arreglos auxiliares, mas copias en memoria, mas lectura de memoria, etc.
Si bien el hecho de medir peaks de memoria para cada ejecucion causa que en el grafico todos los algoritmos utilicen la misma cantidad de memoria extra para su funcionamiento si logramos apreciar como ***merge-sort*** requiere de de memoria extra ($O(n)$) para llevar a cabo el ordenamiento del arreglo.

2.2. Algoritmos de multiplicación de matrices

Introduccion: En esta seccion del experimento realizaremos nuevamente un analisis tanto de tiempo de ejecucion como de uso de memoria, pero en este caso lo realizaremos sobre algoritmos de multiplicación de matrices.
En concreto analizaremos el comportamiento de 2 algoritmos ***naive*** y ***strassen***, siendo el algoritmo naive la opcion mas "*bruta*"para la mutiplicacion ya que consiste en 3 ciclos for anidados, mientras que el algoritmo de strassen sigue el enfoque dividir y conquistar lo que en la teoria reduce la complejidad temporal del mismo.

2.2.1. Analisis de tiempo

	Tamaño	Algoritmo	Tiempo medio de ejecucion	Desviacion estandar	Cantidad de muestras
0	16	naive	184954.777778	14339.648402	18
1	16	strassen	5872281.944444	124022.463514	18
2	64	naive	10862325.777778	125804.566517	18
3	64	strassen	302482287.166667	8508341.955510	18
4	256	naive	730841801.222222	14982636.822267	18
5	256	strassen	15475774095.277779	429441730.023688	18
6	1024	naive	45894626862.722221	257400123.342034	18
7	1024	strassen	760005728144.333374	11907145627.682590	18

Tabla 3: Esta tabla muestra los datos obtenidos de tiempo promedio de ejecucion de los algoritmos para distintos tamaños de entrada.

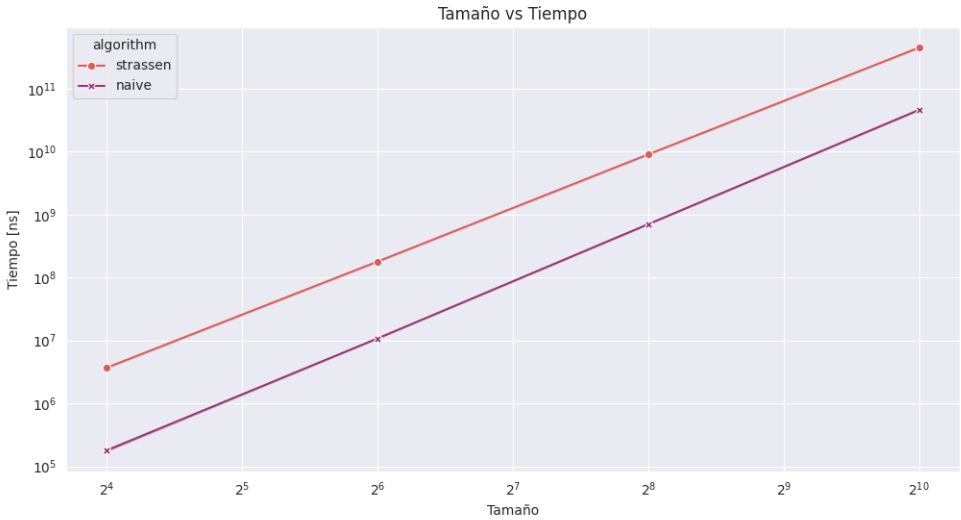

Figura 3: Grafica que relaciona tamaño de entrada con tiempo promedio de ejecucion del algoritmo.

En el grafico anterior podemos apreciar que la tendencia de crecimiento de los algoritmos es polinomial, lo cual es consistente si consideramos las complejidades teoricas para los algoritmos ***naive*** y ***strassen*** de multiplicación de matrices tienen complejidades $O(n^3)$ y $O(n^{\log_2 7}) \approx O(n^{2,81})$ respectivamente. Algo interesante que podemos notar es que para todos los tamaños de entrada el algoritmo ***naive*** tiene tiempos de ejecucion promedio mejores que el algoritmo de ***strassen***.
Esto resulta contradictorio ya que en teoria el algoritmo de ***strassen*** se estipula que este deberia tener un mejor rendimiento, que a pesar de que la mejora no es demasiada conforme el tamaño de la entrada aumenta deberia quedar mas en evidencia esta mejora pero el grafico muestra una realidad completamente distinta.
Este fenomeno se puede deber a que el algoritmo de ***strassen*** en la practica depende fuertemente de la implementacion del mismo, en nuestro caso el uso de estructuras no contiguas tales como `std::vector<std::vector<int>>` para representar las matrices y la creacion de multiples submatrices se traducen en un alto nivel de overhead lo que afecta fuertemente al rendimiento del algoritmo, esto sumado al alto costo por copias y asignaciones dinamicas de memoria causan el efecto observado.
Si bien teoricamente el algoritmo de ***strassen*** deberia ser mas eficiente que el algoritmo ***naive*** ocurre que este ultimo para todos los tamaño analizados en promedio entrega la solucion mas rapido.

2.2.2. Analisis de memoria

	Tamaño	Algoritmo	Uso medio de memoria	Desviacion estandar	Cantidad de muestras
0	16	naive	3692.444444	55.185486	18
1	16	strassen	3670.222222	44.398669	18
2	64	naive	3815.777778	67.610959	18
3	64	strassen	3913.111111	47.185353	18
4	256	naive	5065.111111	49.292511	18
5	256	strassen	6741.111111	50.908094	18
6	1024	naive	24398.222222	52.175671	18
7	1024	strassen	48334.222222	94.887894	18

Tabla 4: Tabla de datos con el promedio de los picos de uso de memoria de cada algoritmo.

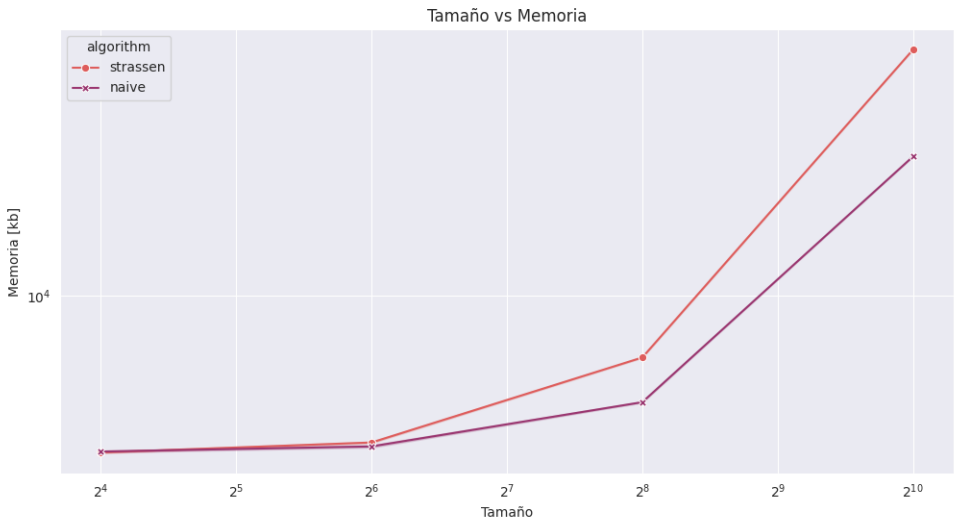

Figura 4: Grafica que relaciona tamaño de entrada con pico de uso de memoria promedio.

En el grafico anterior podemos notar que el uso de memoria para entradas de tamaño 2^4 y 2^6 para ambos algoritmos es practicamente el mismo, esto podria deberse a que en este experimentos se midieron picos de uso de memoria por lo que en este grafico igual se esta considerando el uso de memoria asociado a almacenar la entrada hecha mediante texto a un vector bidimensional.
Sin embargo podemos notar que para los tamaños de entrada 2^8 y 2^{10} el uso de memoria del algoritmo de ***strassen*** crece mas rapido que el uso del algoritmo ***naive***, esto debido a lo mencionado anteriormente en la seccion **??**.
El algoritmo de ***strassen*** realiza muchas mas alocaciones dinamicas de memoria y copias en la misma para realizar la division en submatrices y multiplicarlas para llegar al resultado final. Esto ademas de tener un impacto en el tiempo de ejecucion eleva mucho mas el uso de memoria en comparacion al algoritmo ***naive*** el cual solo hace accesos a memoria mediante indices.

3. Conclusiones

En este trabajo se realizó un análisis experimental de algoritmos de ordenamiento y multiplicación de matrices con el objetivo de contrastar su comportamiento práctico con sus complejidades teóricas. En el caso de los algoritmos de ordenamiento, los resultados obtenidos muestran un crecimiento consistente con una complejidad promedio de ($O(n \log n)$), observándose además que ***std::sort*** y ***quick-sort*** presentan mejores tiempos de ejecución que ***merge-sort***. Esta diferencia se explica principalmente por el hecho de que los primeros son algoritmos in-place y presentan un menor overhead en memoria, lo que impacta positivamente en el rendimiento. Por otro lado, en la multiplicación de matrices, si bien teóricamente el algoritmo de ***strassen*** posee una mejor complejidad que el algoritmo naive, los resultados experimentales muestran que este último es más eficiente para los tamaños de entrada analizados. Esto se debe principalmente a factores de implementación, como el uso de estructuras no contiguas y el costo asociado a la creación de submatrices y operaciones adicionales, lo que introduce un alto overhead. En conjunto, los resultados evidencian que, aunque la teoría permite predecir tendencias generales, el rendimiento real de los algoritmos depende fuertemente de aspectos prácticos como la implementación, el uso de memoria y las características del sistema.